



Linked Lists



Introduction

- Array
 - successive items locate a fixed distance
- disadvantage
 - data movements during insertion and deletion
 - waste space in storing n ordered lists of varying size
- possible solution
 - linked list



4.1 Linked Lists



- list elements are stored, in memory, in an arbitrary order
- explicit information (**called a link**) is used to go from one element to the next

- In previous chapter, representation of **simple data structures**(**Stacks** and **Queue**)using an array and a sequential mapping are discussed.
- These representations had the **property** that successive nodes of the data object were stored a **fixed distance apart**.
 - if the element a_{ij} of a table was stored at location L_{ij} , then $a_{i,j+1}$ was at the location $L_{ij} + c$ for some constant c ;
 - if the i th node in a queue was at location L_i , then the $i + 1$ -st node was at location $L_i + c \bmod n$ for the circular representation;
 - if the topmost node of a stack was at location LT , then the node beneath it was at location $LT - c$, etc.
- However when a **sequential mapping** is used for ordered lists, operations such as **insertion and deletion of arbitrary elements** become **expensive**.

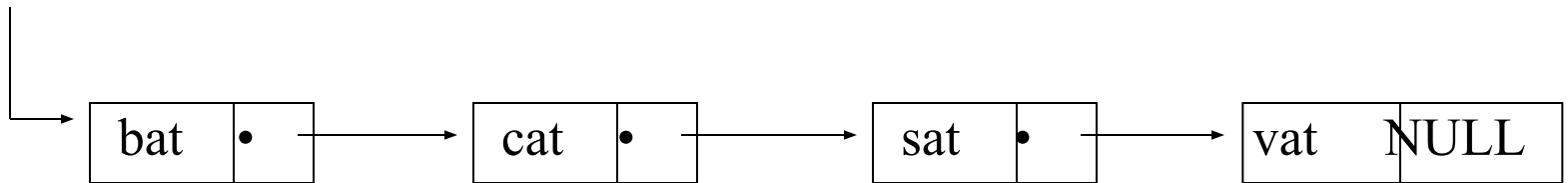
- For example, consider the following list of three letter words:
(BAT, CAT, EAT, FAT, HAT, JAT, LAT, MAT, OAT, PAT, RAT, SAT, TAT, VAT, WAT)
- If an array is used to keep this list, then the insertion of GAT will require us to move elements already in the list either one location higher or lower.
- Move either HAT, JAT, LAT, ..., WAT or else move BAT, CAT, EAT and FAT.
- If many insertions are to be made into the middle, then neither alternative is attractive because of the amount of data movement.
- On the other hand, to remove the word LAT, again we have to move many elements so as to maintain the sequential representation of the list.
- When this problem is called for several ordered lists of varying sizes, sequential representation again proved to be inadequate.
- By storing each list in a different array of maximum size, storage may be wasted.
- By maintaining the lists in a single array a potentially large amount of data movement is needed.

- This was explicitly observed when we represented several **stacks, queues, polynomials and matrices**.
- All these data objects are examples of **ordered lists**.
- **Polynomials** are ordered by **exponent** while **matrices** are ordered by **rows & columns**.
- An alternative representation for ordered lists which will reduce the time needed for arbitrary insertion and deletion is **Linked Lists**.

	ARRAY	LINKED LIST
1.	An array is a grouping of data elements of equivalent data type.	A linked list is a group of entities called a node. The node includes two segments: data and address.
2.	It stores the data elements in a contiguous memory zone.	It stores elements randomly, or we can say anywhere in the memory zone.
3.	In the case of an array, memory size is fixed, and it is not possible to change it during the run time.	In the linked list, the placement of elements is allocated during the run time.
4.	The elements are not dependent on each other.	The data elements are dependent on each other.
5.	The memory is assigned at compile time.	The memory is assigned at run time.
6.	It is easier and faster to access the element in an array.	In a linked list, the process of accessing elements takes more time.
7.	In the case of an array, memory utilization is ineffective.	In the case of the linked list, memory utilization is effective.
8.	When it comes to executing any operation like insertion, deletion, array takes more time.	When it comes to executing any operation like insertion, deletion, the linked list takes less time.

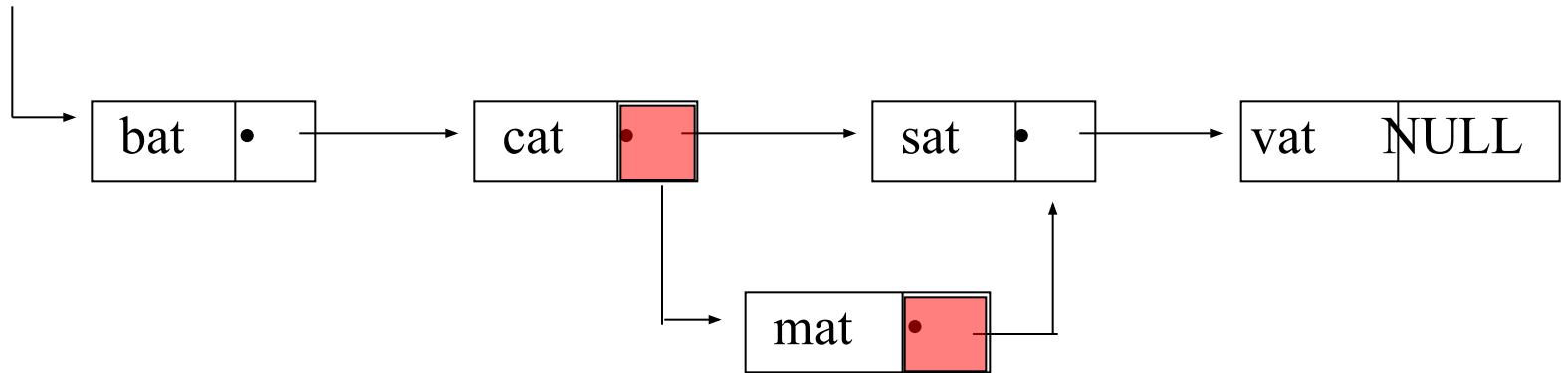
- An elegant solution to the problem of data movement in *sequential* representations is achieved by using *linked representations*.
- In *sequential representation* successive items of a list are located in a *fixed distance apart*, whereas in *linked representation* these items may be placed *anywhere in memory*.
- (i.e) In *sequential representation* the *order of elements* is the *same* as in the ordered list, while in a *linked representation* these *two sequences need not be the same*.
- To access elements in the list in the correct order, with each element we *store the address or location of the next element* in that list.
- Thus, each *data item* in a linked representation is *associated* with a *pointer* to the next item.
- This pointer is often referred to as a *link*.
- In general, a *node* is a collection of data, DATA1, ..., DATA n and links LINK1, ..., LINK m .
- Each item in a node is called a *field*. A field contains either a data item or a link.

SINGLY LINKED LISTS

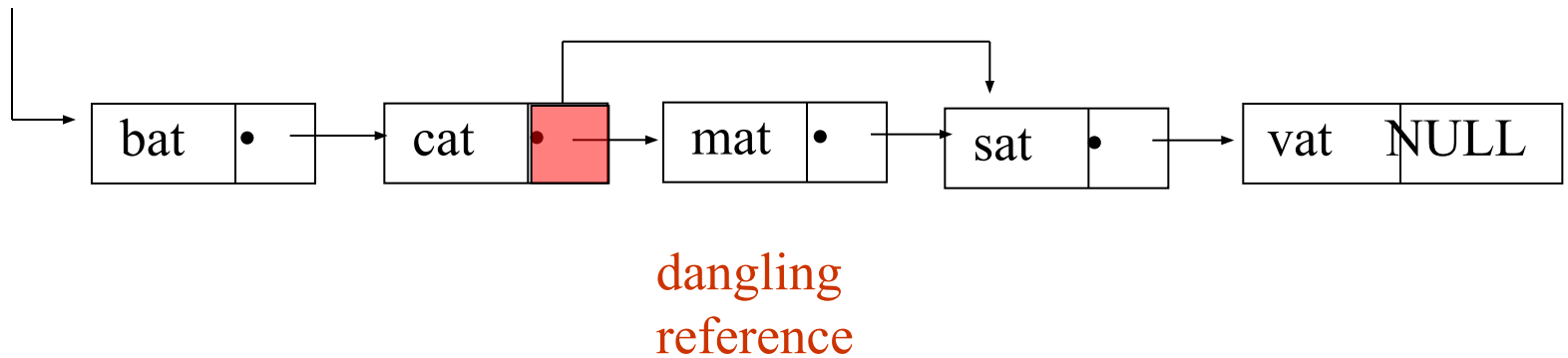


***Figure 4.1: Usual way to draw a linked list (p.139)**

Insertion



***Figure 4.2: Insert mat after cat (p.140)**



***Figure 4.3: Delete *mat* from list (p.140)**

	DATA	LINK
1	HAT	15
2		
3	CAT	4
4	EAT	9
5		
6		
7	WAT	0
8	BAT	3
9	FAT	1
10		
11	VAT	7
	.	.
	.	.
	.	.

Figure 4.1 Non-Sequential List Representation

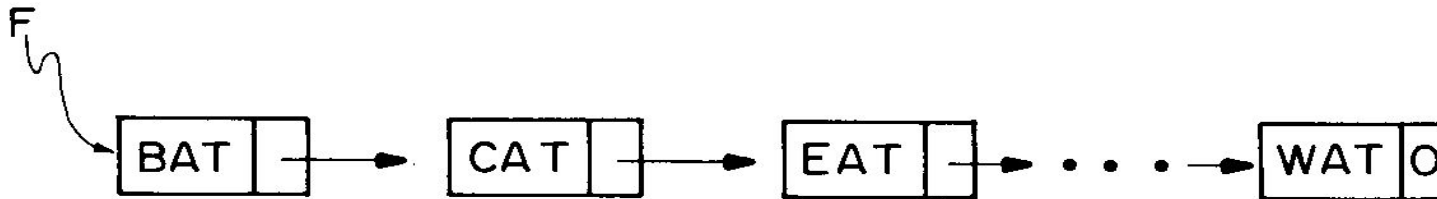


Figure 4.2 Usual Way to Draw a Linked List

Memory Layout

Layout of $L = (a,b,c,d,e)$ using an array representation.

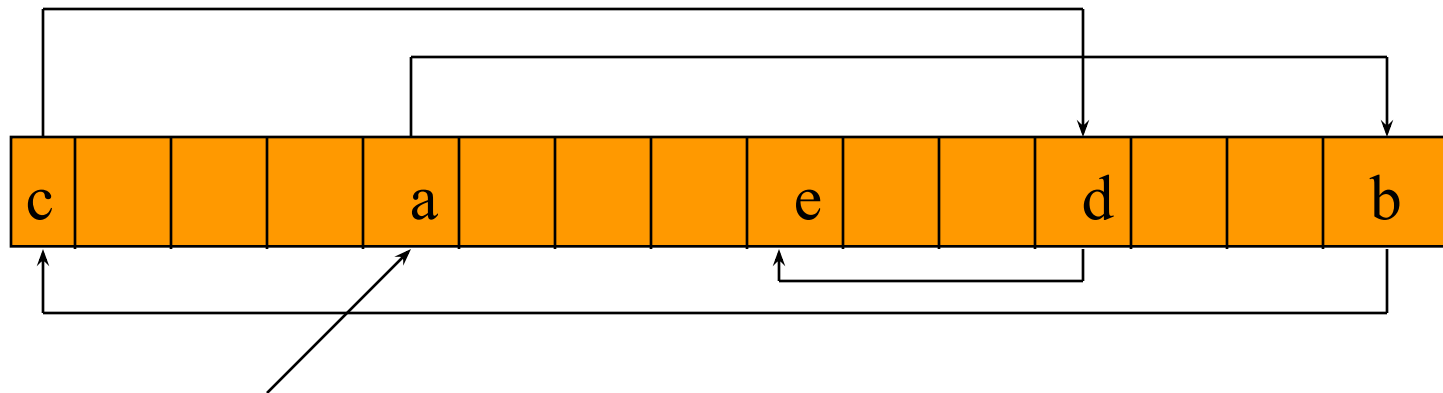


A linked representation uses an arbitrary layout.





Linked Representation

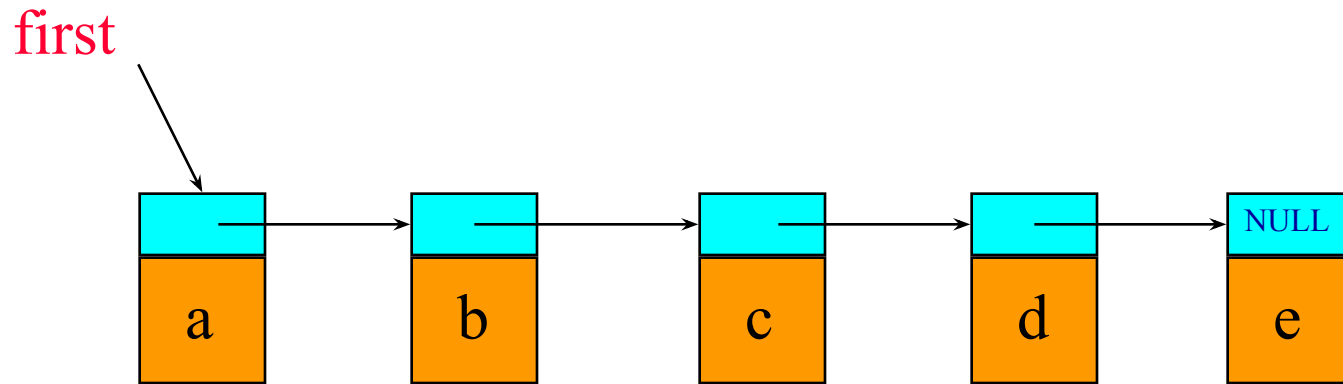


first

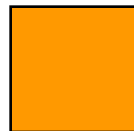
pointer (or link) in **e** is **NULL**

use a variable **first** to get to the first
element **a**

Normal Way To Draw A Linked List

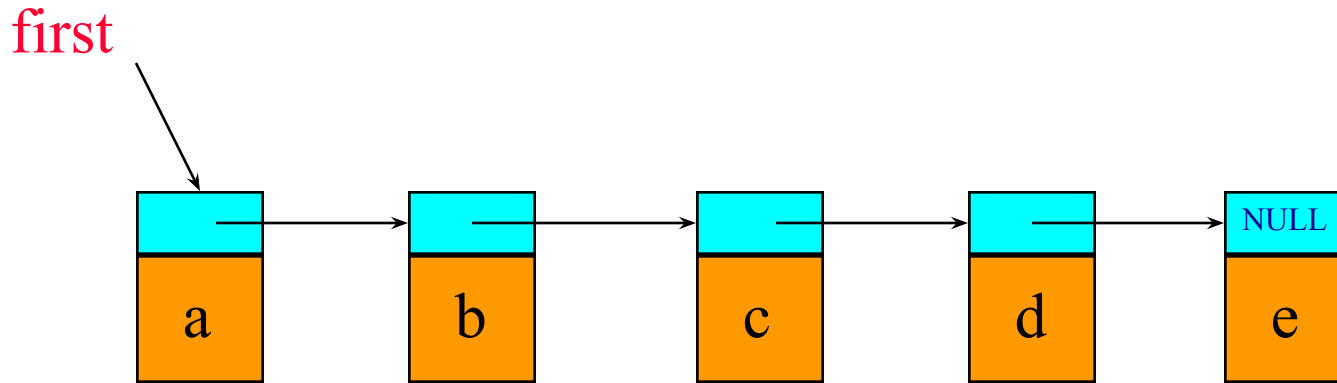


link or pointer field of node



data field of node

Chain



- A chain is a linked list in which each node represents one element.
- There is a link or pointer from one element to the next.
- The last node has a **NULL** (or **0**) pointer.

INSERT

- Arbitrary insertions and deletions are easier using a linked list rather than a sequential list.
- To insert the data item GAT between FAT and HAT the following steps are adequate:
 - get a node which is currently unused; let its address be X ;
 - set the DATA field of this node to GAT;
 - set the LINK field of X to point to the node after FAT which contains HAT;
 - set the LINK field of the node containing FAT to X .
- Fig4.3a shows how the arrays DATA & LINK will be changed after insert GAT.
- Fig4.3b shows insertion process using arrow notation, new arrows are dashed.
- Insertion of GAT do not have to move any other elements in the list.

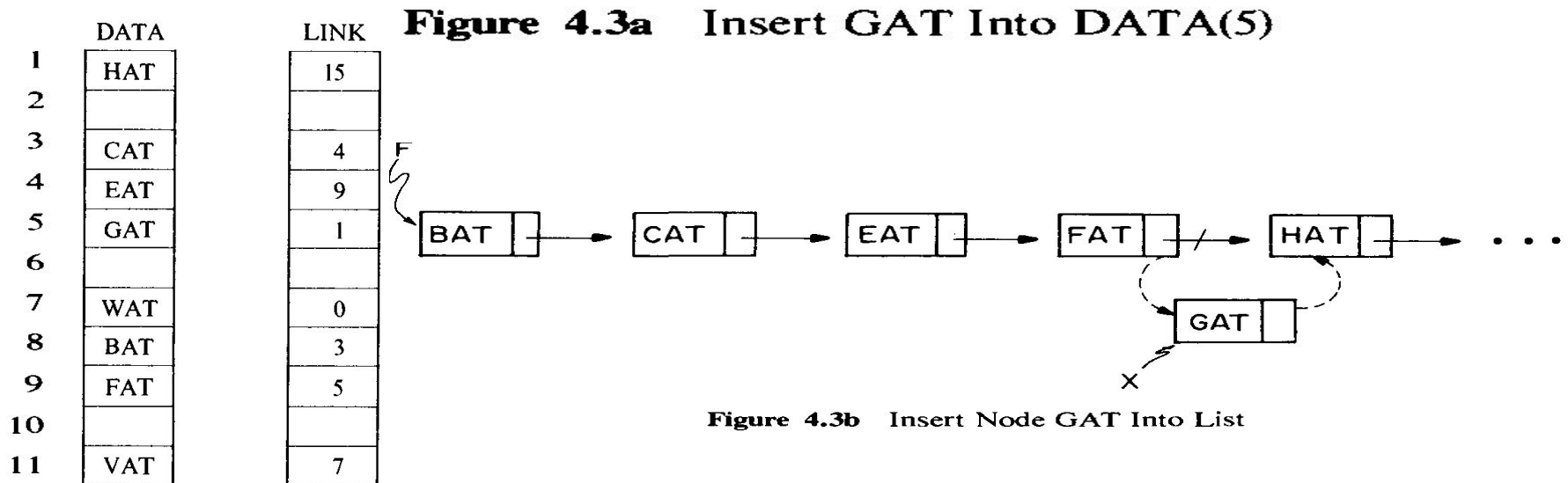


Figure 4.3b Insert Node GAT Into List

DELETE

- To delete GAT from the list, first find the element which immediately precedes GAT, which is FAT, and set LINK(9) to the position of HAT which is 1.
- Again, there is no need to move the data around.
- Even though the LINK field of GAT still contains a pointer to HAT, GAT is no longer in the list (see figure 4.4).

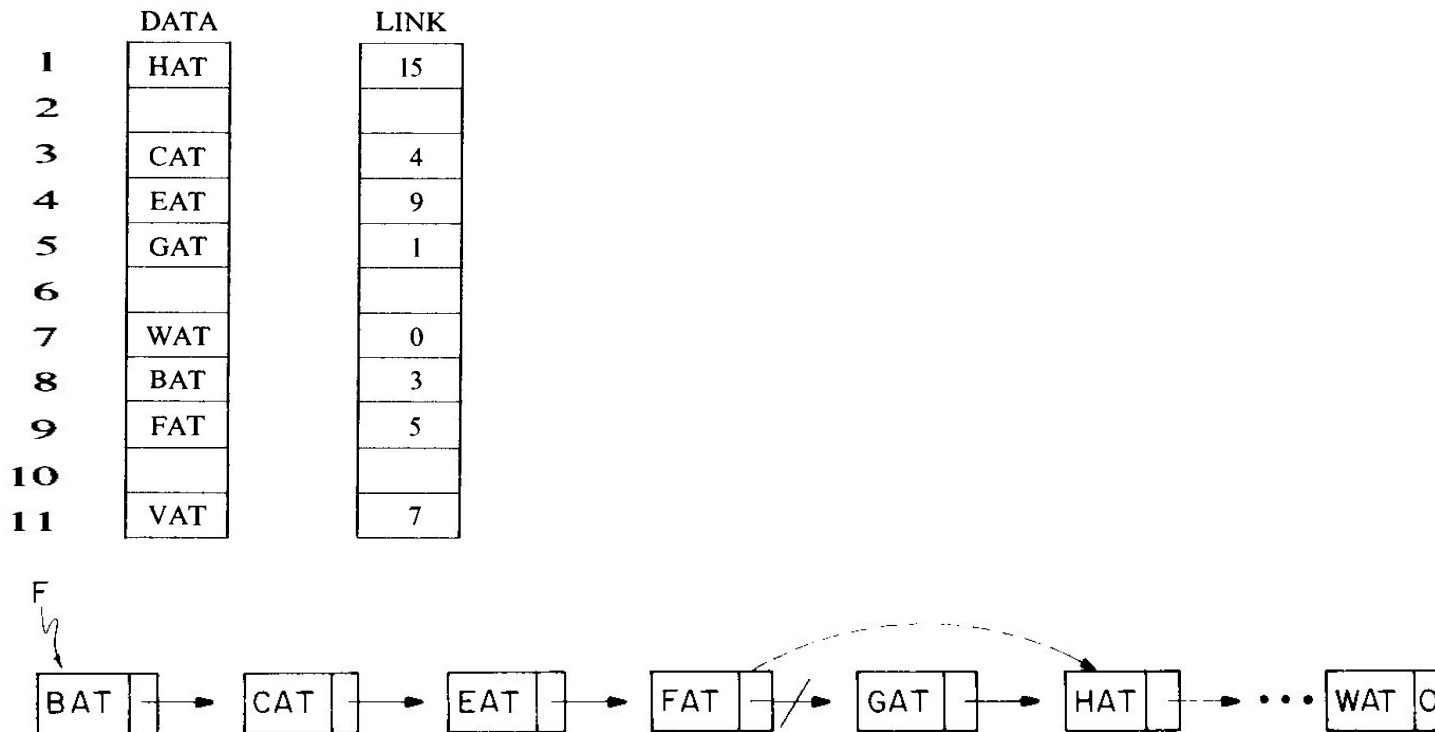


Figure 4.4 Delete GAT from List

- From the brief discussion of linked lists, the following capabilities are needed to make linked representations possible:
 - A means for dividing memory into nodes each having at least one link field;
 - A mechanism to determine which nodes are in use and which are free;
 - A mechanism to transfer nodes from the reserved pool to the free pool and vice-versa.
- Let us assume that all free nodes are kept in a "black box" called the storage pool and that there exist subalgorithms:
 - GETNODE(X) which provides in X a pointer to a free node but if no node is free, it prints an error message and stops;
 - RET(X) which returns node X to the storage pool.

CREATE

- **Example 4.1:** Assume that each node has two fields DATA and LINK.
- The following algorithm creates a linked list with two nodes whose DATA fields are set to be the values 'MAT' and 'PAT' respectively.
- T is a pointer to the first node in this list.

procedure *CREATE2*(T)

call *GETNODE*(I) //get an available node//

$T \leftarrow I$; $DATA(I) \leftarrow 'MAT'$ //store information into the node//

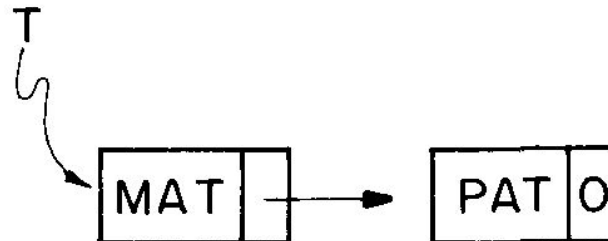
call *GETNODE*(I) //get a second available node//

$LINK(T) \leftarrow I$ //attach first node to the second//

$LINK(I) \leftarrow 0$; $DATA(I) \leftarrow 'PAT'$

end *CREATE2*

The resulting list structure is

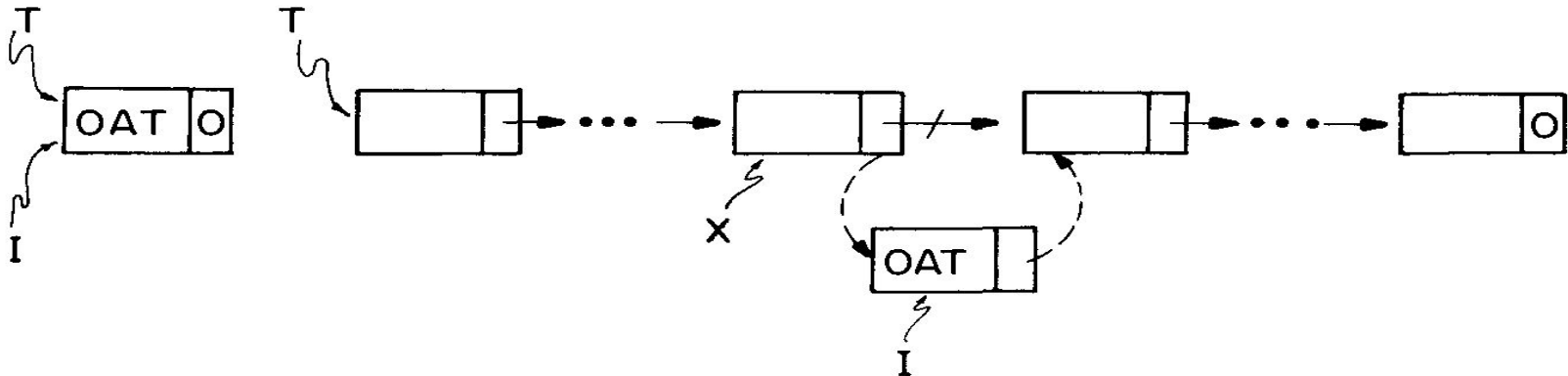


INSERT

- **Example 4.2:** Let T be a pointer to a linked list as in Example 4.1.
- $T = 0$ if the list has no nodes.
- Let X be a pointer to some arbitrary node in the list T .
- The following algorithm inserts a node with DATA field 'OAT' following the node pointed at by X .

```
procedure INSERT( $T, X$ )  
  call GETNODE( $I$ )  
  DATA( $I$ )  $\leftarrow$  'OAT'  
  if  $T = 0$  then [ $T \leftarrow I$ ; LINK( $I$ )  $\leftarrow$  0]           //insert into empty list//  
    else [ $LINK(I) \leftarrow LINK(X)$                          //insert after  $X$ //  
          LINK( $X$ )  $\leftarrow I$ ]  
end INSERT
```

The resulting list structure for the two cases $T = 0$ and $T \neq 0$ is



DELETE

- **Example 4.3:** Let X be a pointer to some node in a linked list T as in example 4.2.
- Let Y be the node preceding X .
- $Y = 0$ if X is the first node in T (i.e., if $X = T$).
- The following algorithm deletes node X from T .

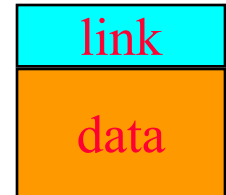
```
procedure DELETE( $X, Y, T$ )  
  if  $Y = 0$  then  $T \leftarrow LINK(T)$       //remove the first node//  
    else  $LINK(Y) \leftarrow LINK(X)$       //remove an interior  
                                         node//  
  call RET( $X$ )      //return node to storage pool//  
end DELETE
```

EXTRA SLIDES

- Diagrams - linked list operations

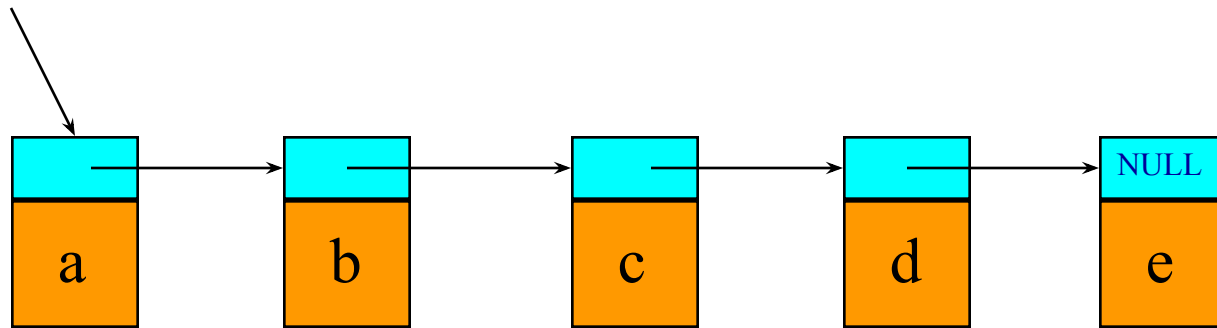
Node Representation

```
typedef struct listNode *listPointer;  
typedef struct {  
    char data;  
    listPointer link;  
} listNode;
```



get(0)

first



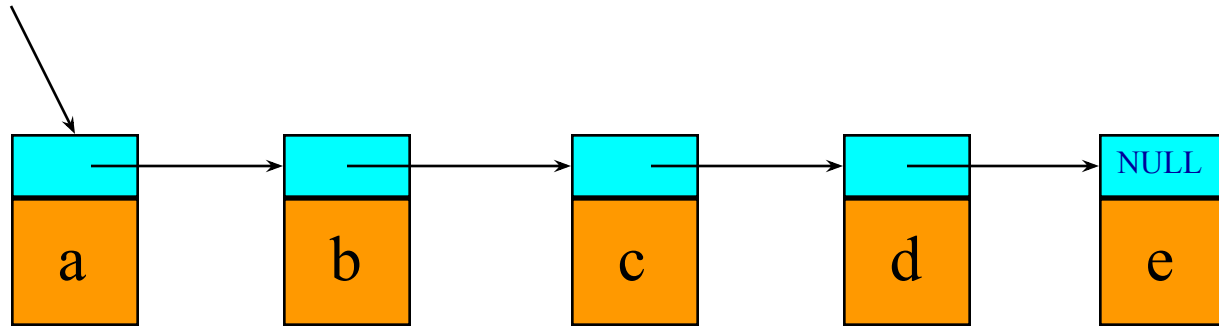
`desiredNode = first; // gets you to first node – a node`

`return desiredNode->data; a`

When `firstNode = null`, there is no element whose index is 0.

get(1)

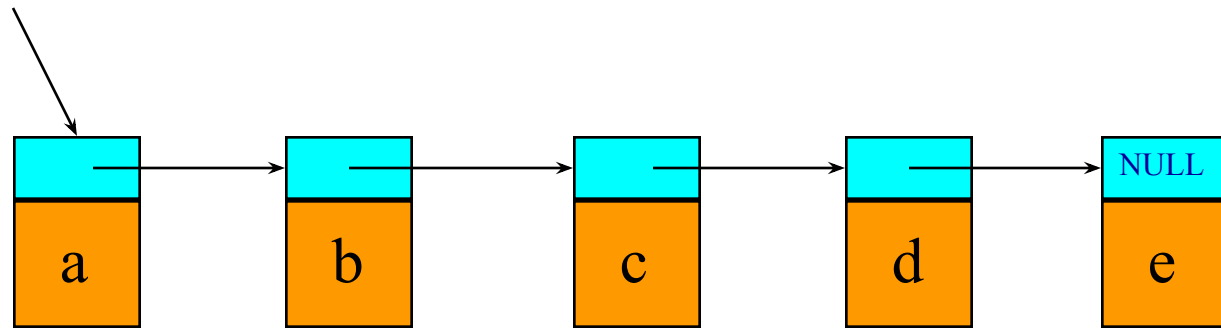
first



```
desiredNode = first->link; // a □ b gets you to second node, b node  
return desiredNode->data;  // b
```

get(2)

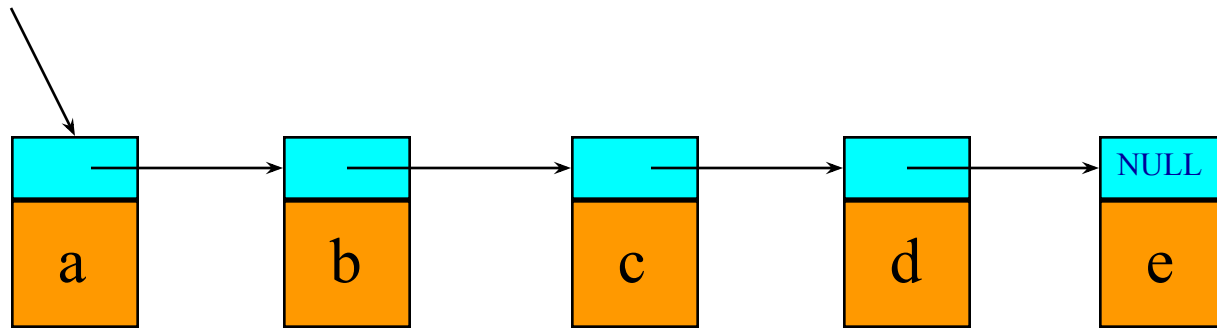
first



```
desiredNode = first->link->link; // a □ b □ c gets you to third node/c node  
return desiredNode->data;         c
```

get(5)

first

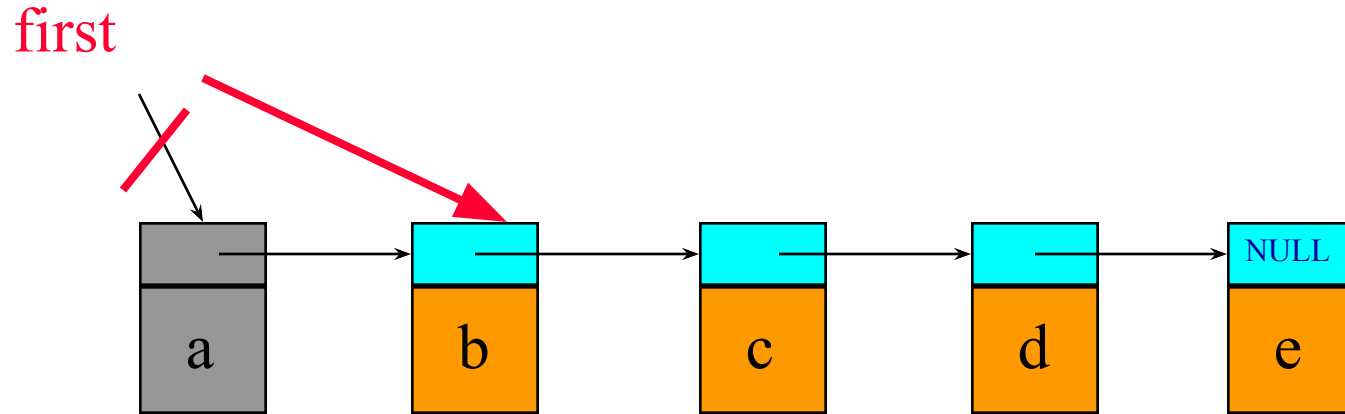


```
desiredNode = first->link->link->link->link->link;
```

```
//a □ b □ c □ d □ e □ Null desiredNode = NULL
```

```
return desiredNode->data; // NULL.element
```

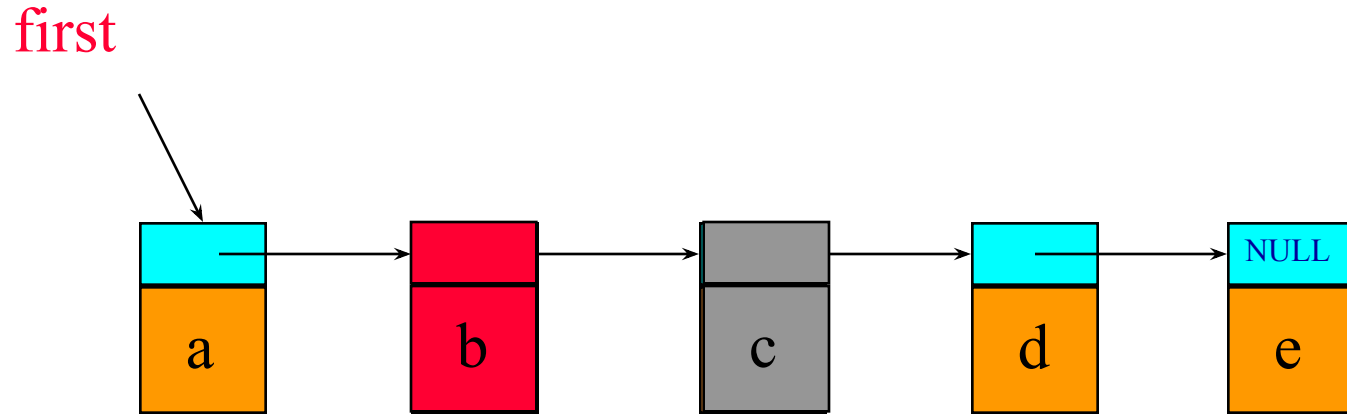
Delete An Element



delete(0)

```
deleteNode = first; //a node  
first = first->link; // b node  
free(deleteNode); // a
```

delete(2)

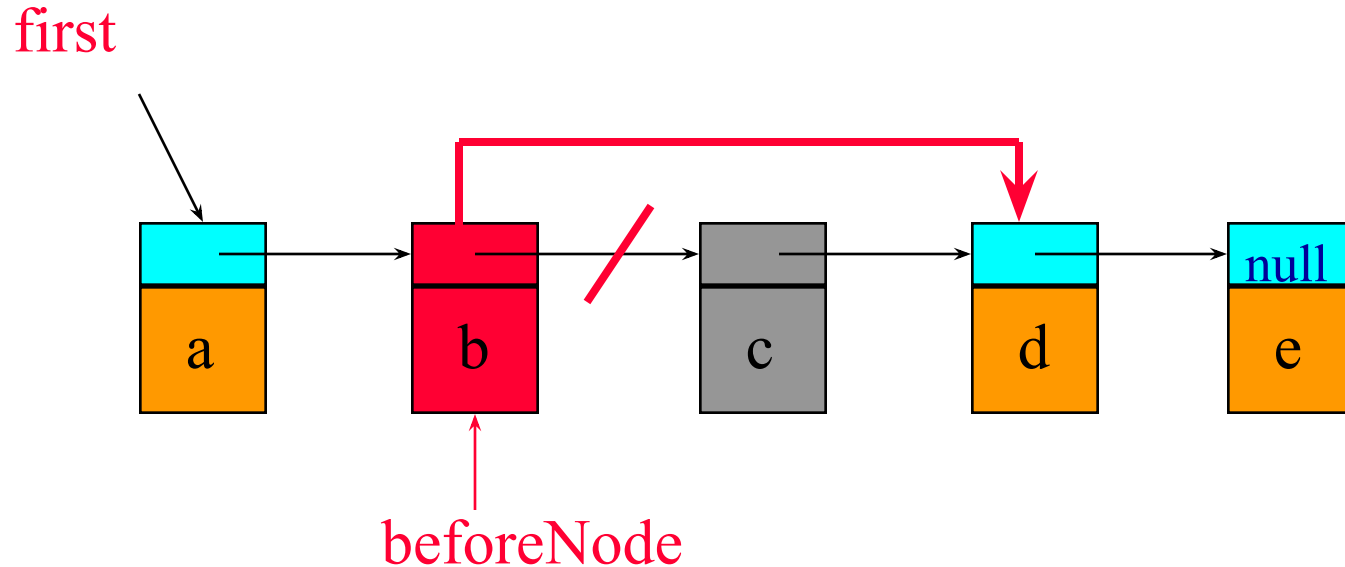


beforeNode

first get to node just before node to be removed

`beforeNode = first->link; // b node`

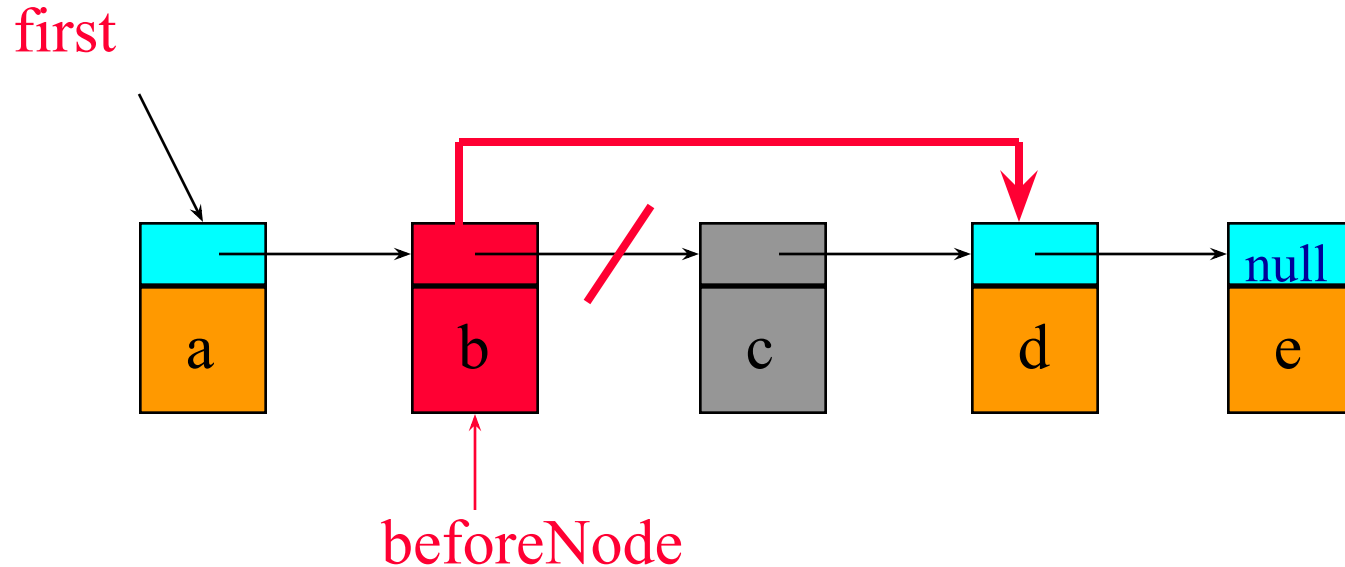
delete(2)



save pointer to node that will be deleted

```
deleteNode = beforeNode->link;  
//b node □ link // c node
```

delete(2)



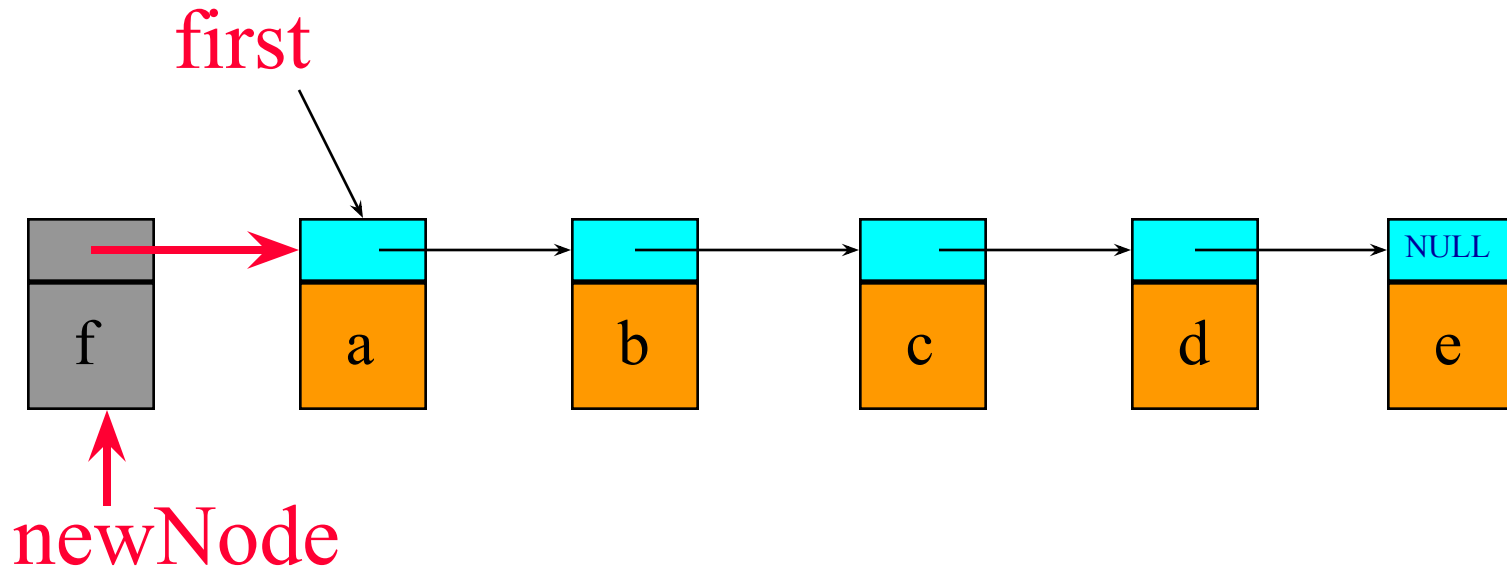
now change pointer in **beforeNode**

```
beforeNode->link = beforeNode->link->link;
```

```
// bnode □ link □ link // d node.
```

```
free(deleteNode); // c node
```


insert(0, 'f')



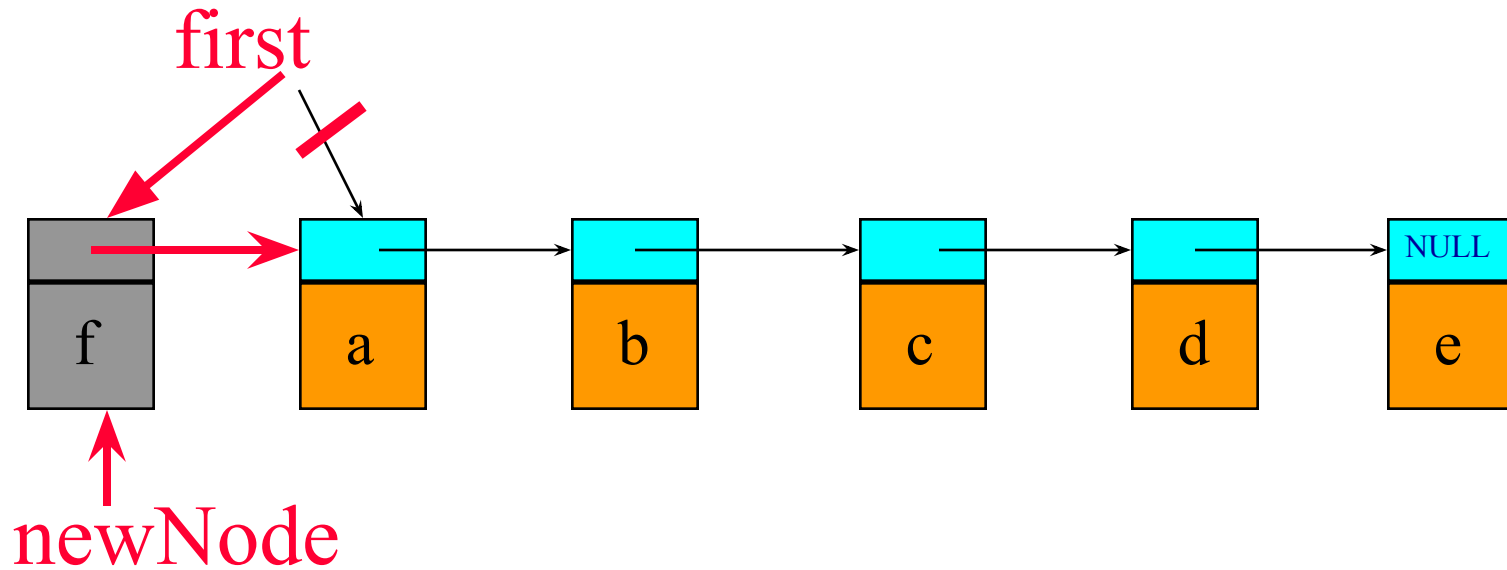
Step 1: get a node, set its data and link fields

```
MALLOC( newNode, sizeof(*newNode));
```

```
newNode->data = 'f';
```

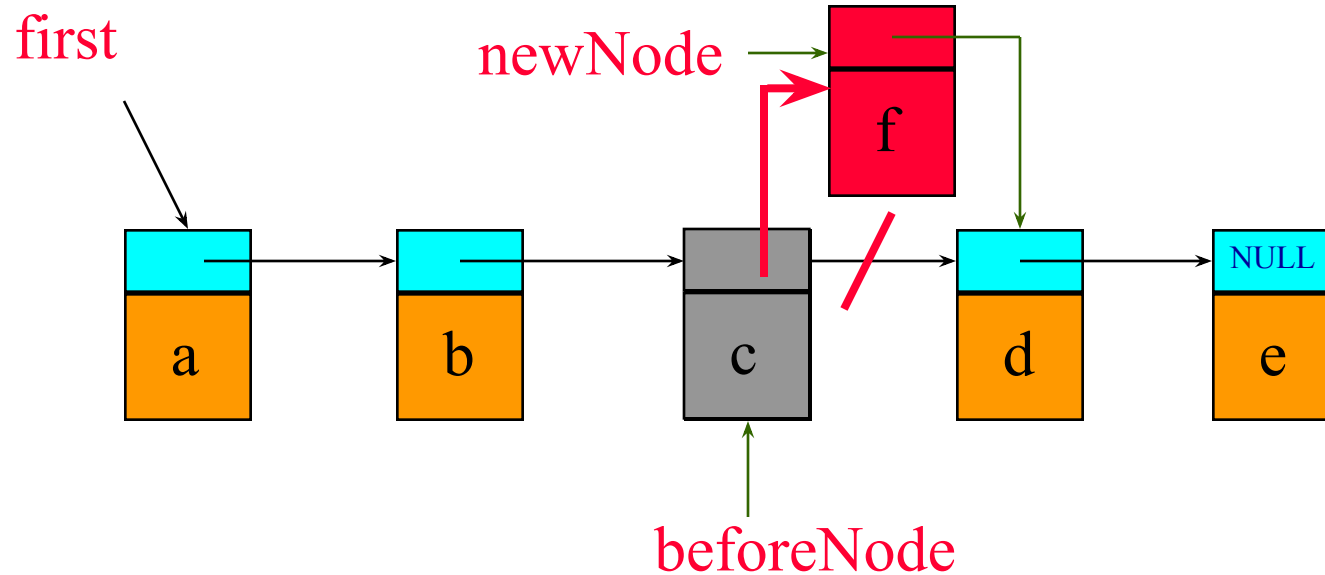
```
newNode->link = NULL;
```

insert(0, 'f')



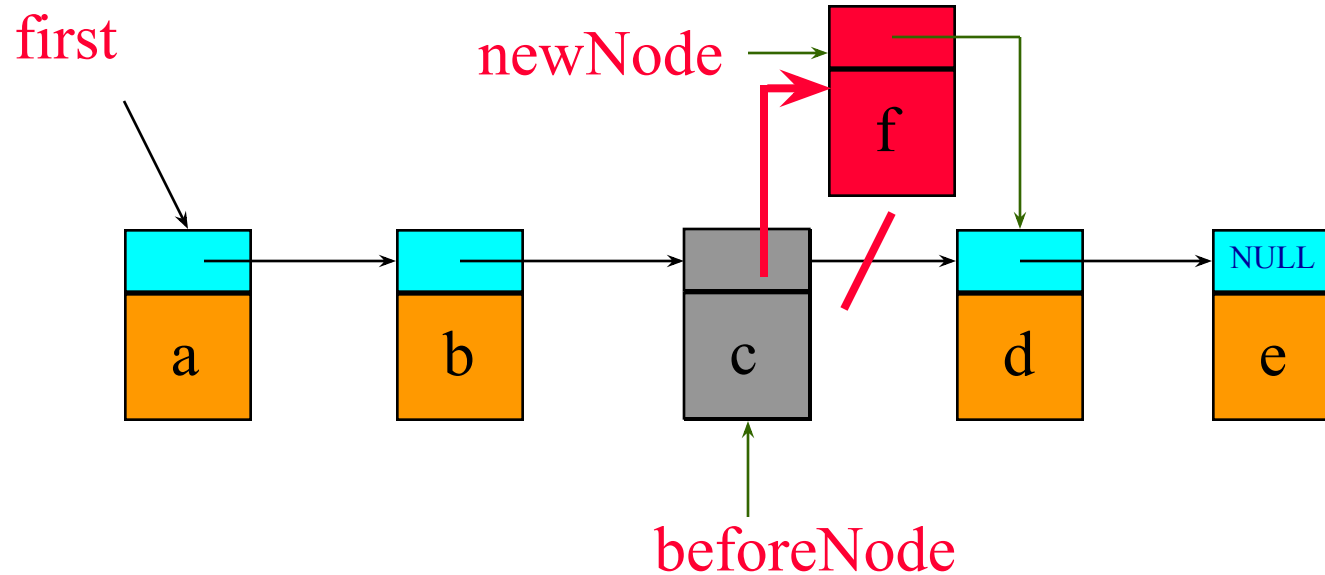
Step 2: update **first**
first = newNode;

insert(3, 'f')



- first find node whose index is 2
- next create a new node and set its data and link fields
- finally link **beforeNode** to **newNode**

insert(3, 'f')



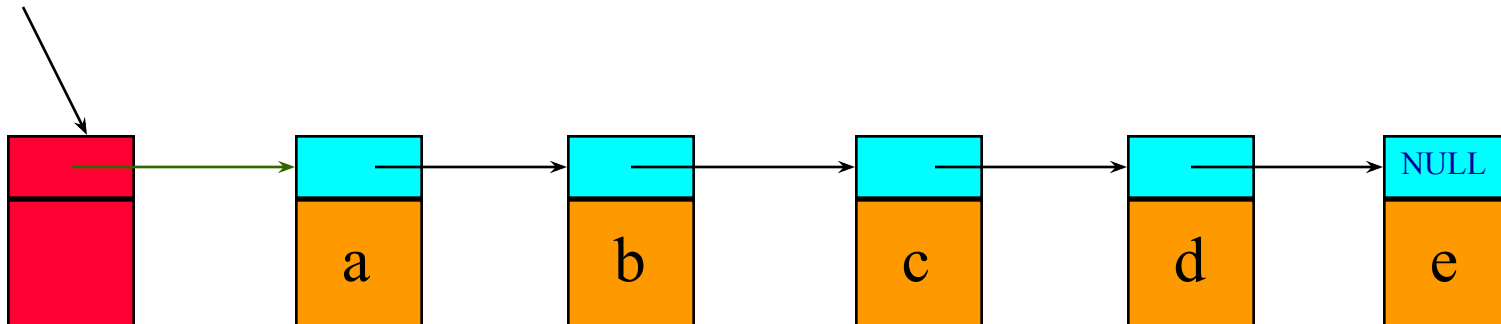
```
beforeNode = first->link->link; // a □ b □ c  
MALLOC( newNode, sizeof(*newNode));  
newNode->data = 'f';  
newNode->link = beforeNode->link;  
    // f node □ link = c node □ link // d node  
beforeNode->link = newNode; // c node □ link = f node
```



Chain With Header Node

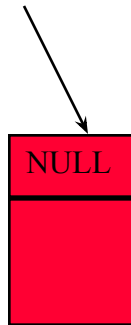


headerNode



Empty Chain With Header Node

headerNode

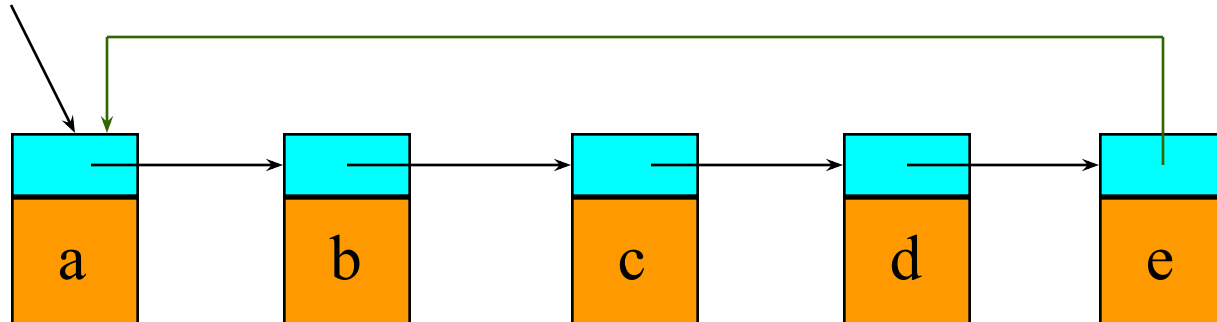




Circular List



firstNode





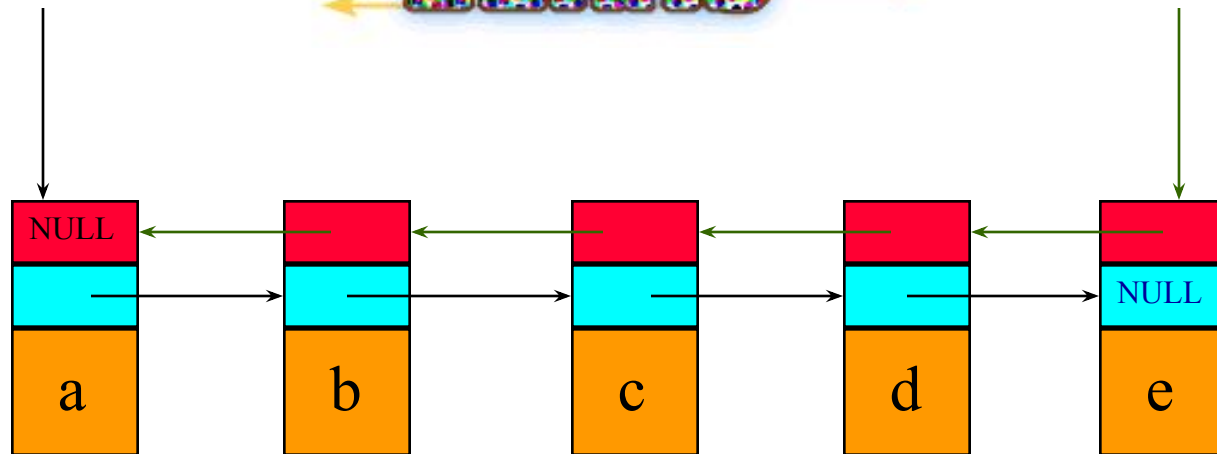
Doubly Linked List



firstNode



lastNode



Can be used to reduce worst-case run time of a linear list
add/remove/get by half.

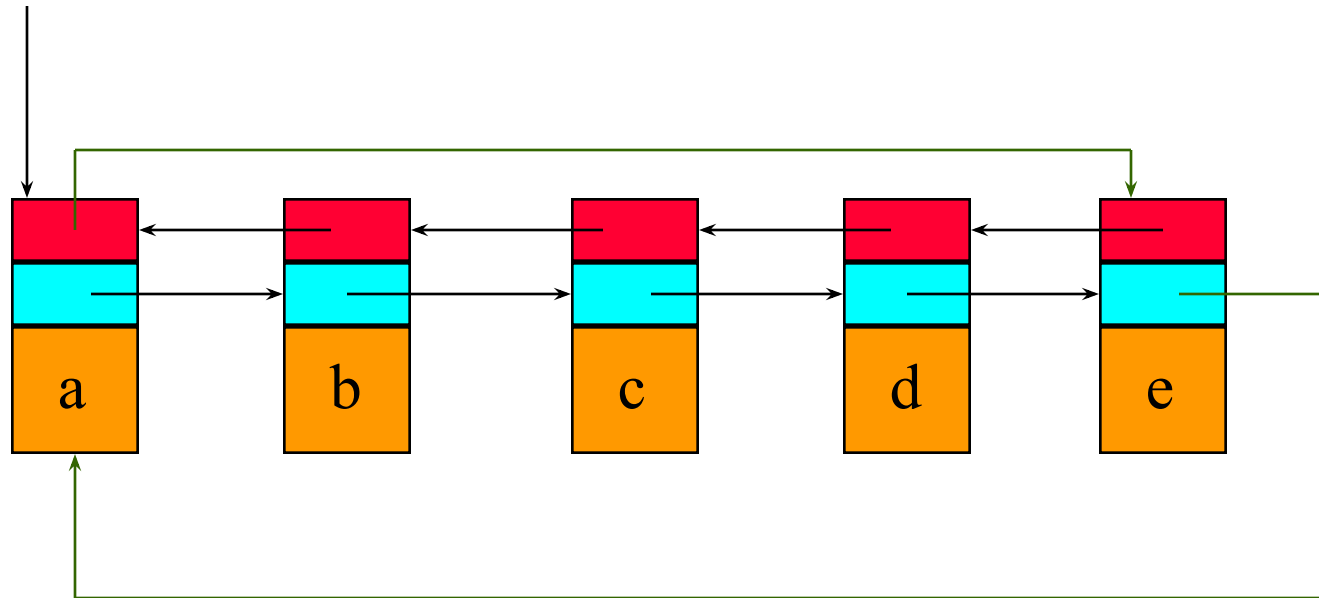
Start at left end if $\text{index} \leq \text{listSize}/2$; otherwise, start at right end.



Doubly Linked Circular List



firstNode

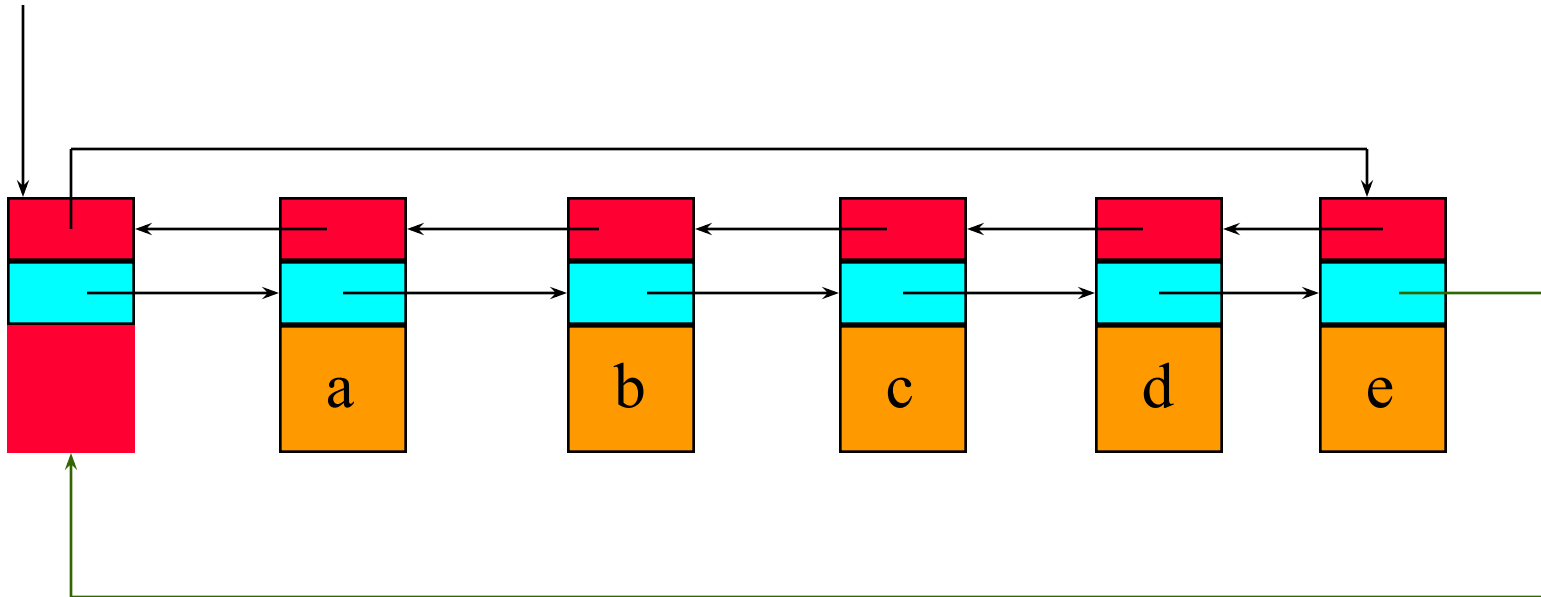




Doubly Linked Circular List With Header Node



headerNode



Empty Doubly Linked Circular List With Header Node



headerNode

